EMBEDDED SYSTEMS PORTFOLIO

Greenhouse monitoring and control

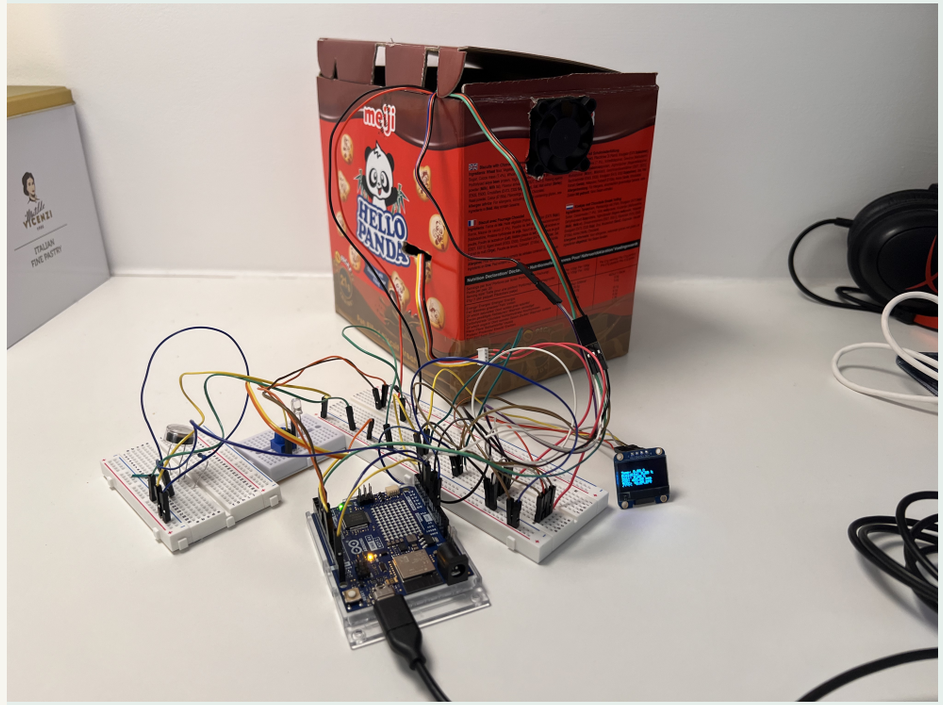
A model greenhouse with local sensing, actuation and a browser interface.

I designed and assembled a model greenhouse controller around an Arduino UNO R4 WiFi, combining sensor acquisition, fan switching, LED brightness control, an OLED and a local browser interface.

4
sensor modules

6
reading channels

2
actuation types



Original prototype / April 2025 report

C/C++ / Analogue and I2C integration / HTTP / Bench testing

Individual project
Chi Zou | 2024-2025

S1 pp. 6-18, 23-25; S2-S3; original prototype photo in S6.

01 / INTEGRATION

Four sensor modules and one shared state

The firmware connects six readings to control decisions and two user interfaces.

Acquire
zc.ino / analogue, digital and I2C

→

Share state
MyParameters.cpp / readings + targets

→

Act and expose
MyControl.cpp / OLED / HTTP

DHT11	D5	Temperature and air humidity
LDR	A0	1023 - analogRead(A0)
Soil probe	A1	map(ADC, 0, 1023, 0, 100)
SGP30 and OLED	I2C	eCO2 and TVOC / local display
LED and fan outputs	D11 / D9, D6	PWM lighting / threshold switching

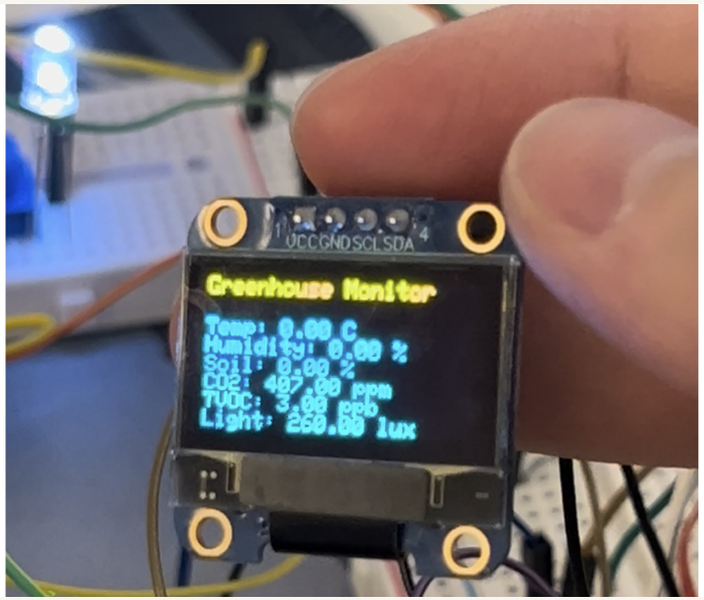
Firmware pin map shown. Light and soil values are indicators; units require calibration.

S2 zc.ino lines 13-26, 109-138; MyParameters.cpp lines 4-5; S1 pp. 6-7, 12, 14-18.

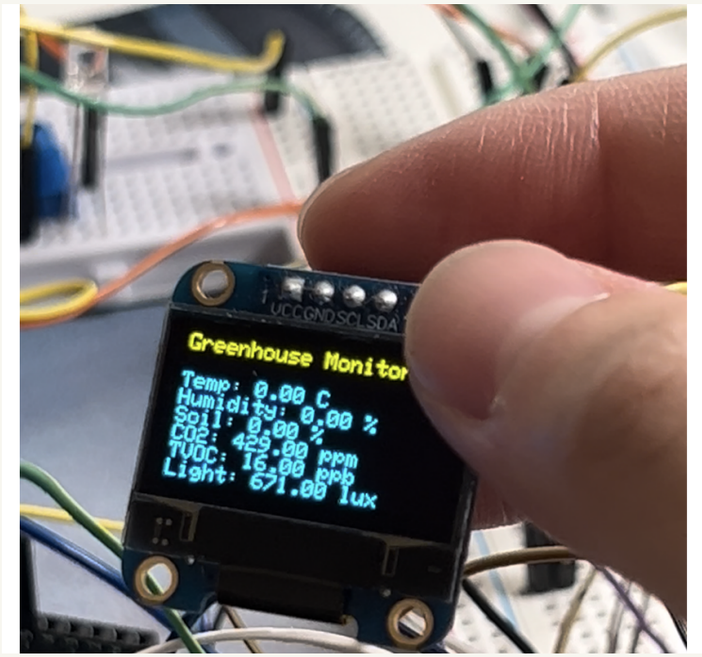
02 / OBSERVATION

A light reading becomes an actuator decision

Two recorded states connect the displayed input to the visible LED response.



260 / LED ON



671 / LED OFF

Recorded values from the report

500

configured light threshold

Observe the complete path

Input on the OLED and LED state appear in the same frame. The pair demonstrates a response across the threshold.

Two recorded states; no measured response-time or energy-saving claim.

Original OLED labels say lux. Firmware shows an uncalibrated ADC-derived light indicator.

S1 pp. 21-22, Fig. 14; S6 LEDon.png and LEDoff.png; S2 MyParameters.cpp line 5; zc.ino line 128.

03 / CONTROL DESIGN

Explicit control rules make behaviour inspectable

Lighting uses a bounded PWM update; ventilation uses a threshold OR rule.

Lighting / bounded PWM

```
if (lightIntensity < targetLight)
    step = +10;
else
    step = -10;
pwm = clamp(pwm + step, 0, 255);
```

Equivalent pseudocode, from MyControl.cpp.

26 control calls

From PWM 0 to 255 under sustained low light. This count is derived from code; elapsed time depends on the complete loop.

Ventilation / threshold OR

```
fan_on = (temperature > 30)
      || (airHumidity > 60)
      || (co2 > 600)
      || (tvoc > 200);
```

Defaults shown: deg C / %RH / ppm eCO2 / ppb.

One rule for four channels

Both fan outputs switch together. The soil target is stored and displayed, but it has no actuator branch in this snapshot.

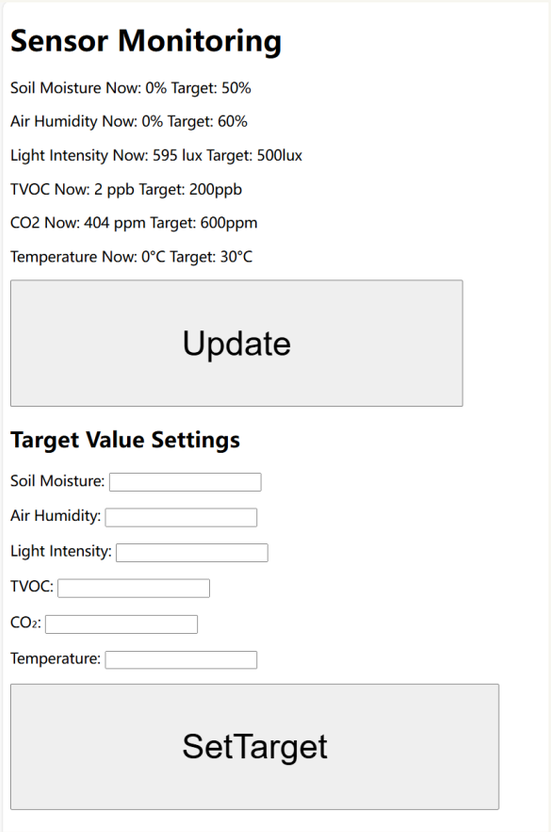
Source review: equality reduces LED PWM; no hysteresis band is implemented.

S2 MyControl.cpp lines 10-24, 34-46; MyParameters.cpp line 5; zc.ino lines 71-105. Saturation count = ceil(255/10).

04 / LOCAL COMMUNICATION

A browser can inspect and change the controller

The board hosts a local access point and HTTP server without a cloud service.



Original browser capture

GET /data

6 readings + 6 targets -> JSON -> browser display

POST /set?key=value

Browser form -> parser -> shared targets -> control logic

1000 ms

configured browser polling interval

The access point and HTTP server run locally. Web commands change targets; they do not start the five-second serial override timeout.

The screenshot preserves the original labels. Polling is a code setting, not a latency benchmark.

S3 MyWIFI.cpp lines 11-19, 23-95; HTML.cpp lines 10-60; original interface capture in S6.

05 / TESTABILITY

Separate test inputs from operating targets

Serial value injection makes control conditions accessible during bench testing.

Live readings
Sensors feed the shared values

→

Injected fields
Serial input sets per-field flags

→

Return to live data
Elapsed > 5000 ms at loop check

Existing input examples

```
temp=50
humidity=70
co2=700
light=700
```

Each line is a separate example from parseinput.txt.

A deliberate test interface

Substituting a reading can exercise a control condition without waiting for the room environment to change. Only fields with an active flag are replaced.

Next boundary cases

Check below, equal and above target; repeat a command; send malformed input; observe recovery. Save actuator output with the injected value.

Code path inspected. The proposed boundary tests have not been run as part of this portfolio.

S2 zc.ino lines 81-89, 109-138, 177-231; Finalreport/Code/parseinput.txt.

06 / MEASUREMENT PRACTICE

Sensor acquisition with explicit timing

A dedicated Arduino test and Python visualisation script expose the measurement path.

DHT11 on D5

DHT.read() -> humidity + temperature + status

Serial at 9600 baud

Time, status, humidity and temperature fields

PySerial and Matplotlib

Parse incoming values -> append -> plot

Acquisition and plotting diagram derived from the saved source.

Status codes are printed by the sketch. The Python parser does not filter failed status rows before plotting.

300 s

configured test duration

2000 ms

delay in acquisition sketch

1000 ms

Python plot refresh setting

The saved scripts establish the acquisition workflow. The reported comparison curve is assumed, so it cannot establish calibration accuracy.

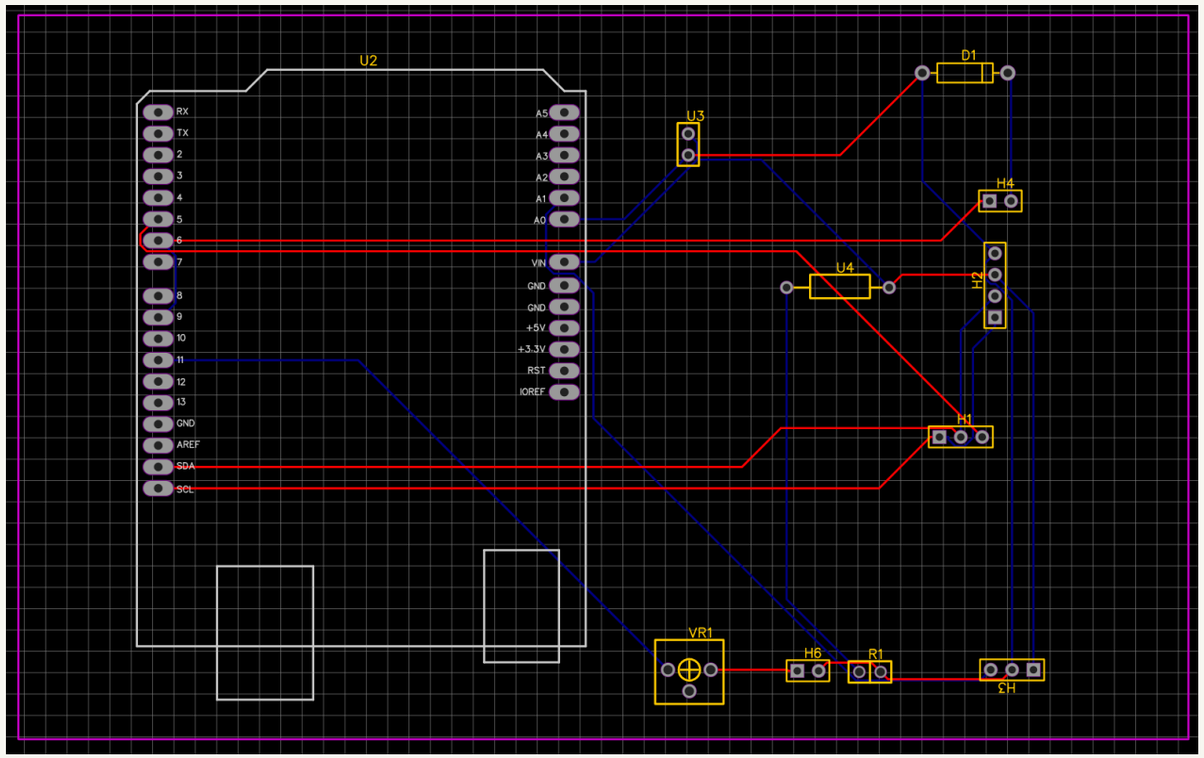
Next: retain raw timestamps and status, then compare against an independent reference.

S1 pp. 19-21; S4 DHT-11G.ino and import serial.py; S5 section 3.6 and Appendix 5. Diagram redrawn from saved code.

07 / HARDWARE DEVELOPMENT

Carry the prototype into a reviewable PCB design

The archive preserves the physical assembly, circuit drawings and board layout.



Saved PCB layout / design stage

Design evidence

I assembled the breadboard prototype and documented the circuit by module. A saved PCB layout and editable CAD files show the intended next hardware form. These demonstrate design work, rather than a manufactured and tested PCB.

An explicit pin contract

Drawing and code agree: LED D11 / soil A1 / LDR A0 / DHT11 D5 / fan D9 and D6. Freeze the map with the board revision before bring-up.

The interim report describes a 5 V fan spin test. Rated supply, driver current and airflow remain separate checks.

S1 pp. 23-25; S5 section 3.7; S7 circuit figures and CAD files; S2 zc.ino lines 17-23.

08 / ENGINEERING JUDGEMENT

Define the evidence needed for the next iteration

The prototype demonstrates integration. These tests would establish stronger operating guarantees.

01

Measurement validity

Tie physical units to calibration. Save reference readings beside sensor values and flag stale samples. The integrated sketch converts DHT readings to integers before isnan checks; I would preserve floating-point validity until the checks complete.

02

Control timing and stability

The loop includes serial and HTTP operations and has no active CBTimer schedule. I would log loop time under browser traffic, set a deliberate control period and measure behaviour around thresholds before adding hysteresis.

03

Hardware release

Freeze a versioned pin map, verify the fan driver and supply, review the layout, then bring up a manufactured board in stages. Calibration, timing measurements and board tests are proposed validation, not achievements claimed for this prototype.

S2 zc.ino lines 71-138; S3 request handling; S7 schematic comparison. Review observations and proposed validation, October 2026.

09 / TRACEABILITY

Evidence index and project credits

Every result and diagram in this case has a stated origin.

S1 Final project report

Finalreport/Final Report.pdf, dated 9 April 2025. Architecture pp. 6-7; software pp. 14-19; experiments pp. 19-22; assembly and PCB pp. 23-25. Page numbers are PDF page numbers.

S2 Firmware and S3 browser interface

Finalreport/Code/zc/: zc.ino, MyControl.cpp, MyParameters.cpp, MyWIFI.cpp and HTML.cpp. The uncompressed directory is the snapshot used here; zipped and earlier test copies differ.

S4 Acquisition and S5 process record

S4: test-code folder, modified DHT-11 test, DHT-11G.ino and import serial.py. S5: Interim Report/Interim Report-Greenhouse(Chi Zou).docx and project_proposal_zc.docx.

S6 Original images and S7 hardware files

S6: Finalreport/ultimate device.jpg; Analysis graph/LEDon.png, LEDoff.png and website data.png; Interim Report/DHT-11.png. S7: Finalreport/Modular circuit diagram/; PCBdesign/PCB.png and PCB.dsn; zcpro/ CAD JSON files.

Attribution and verification

Individual UCL EEE project by Chi Zou, supervised by Dr Oliver Hadeler. Sensor, display and Wi-Fi libraries are third-party dependencies. CBTimer credits embedded-kiddie; it is not scheduled in the integrated sketch. The interim report acknowledges AI assistance for plotting. Saved evidence was reviewed; hardware tests were not rerun.

Code repository recorded in the final report

github.com/LuciferZC/Third-Year-Project